

LegalEase

AI-Powered Legal Document Generator

NAAN MUDHALVAN PROJECT

Detailed Project Documentation

Group Project | Team Leader: Guna K | Team Size: 5

Developed using Streamlit, FastAPI, Google Gemini and Python

Table of Contents

1. Project Overview
2. Project Scenarios and Use Cases
3. Pre-requisites
4. Project Workflow
5. Milestone 1 – Model Selection and System Architecture
6. Development Environment and Project Structure
7. Milestone 2 – Core Functionalities
8. Document Formatting and Export
9. Milestone 3 – API Logic Integration
10. Backend Components
11. Milestone 4 – Frontend Development
12. Dynamic Preview and Editing
13. AI Integration and Prompting
14. Milestone 5 – Deployment
15. Testing and Verification
16. Exploring LegalEase – User Inputs
17. Generated Document and Downloads
18. Security and Environment Configuration
19. Deployment Architecture
20. Team Contribution
21. Limitations and Future Enhancements
22. Conclusion
23. Disclaimer

1. Project Overview

LegalEase is an AI-powered legal document generation application designed to simplify the creation of structured legal document drafts. The project combines a Streamlit frontend, a FastAPI backend and Google Gemini-based generation to turn user-provided document details into an editable draft.

The source project description identifies common document use cases such as employment contracts, lease agreements and non-disclosure agreements. The application is designed around inputs including document type, parties involved, terms and effective dates.

The implemented application extends this workflow with professional document formatting, LegalEase branding, editable previews and downloads in TXT, DOCX and PDF formats.

Project Goals

- Generate structured legal document drafts from user-provided information.
- Keep the generated result editable before export.
- Provide multiple output formats for practical use.
- Maintain consistent document formatting and branding.
- Connect a simple web interface with a modular API and AI core.

Project Identity

Item	Details
Project	LegalEase: AI-Powered Legal Document Generator
Programme	NAAN MUDHALVAN
Project Type	Group Project
Team Size	5
Team Leader	Guna K
Frontend	Streamlit
Backend	FastAPI
AI	Google Gemini
Language	Python

2. Project Scenarios and Use Cases

The original project documentation describes three representative scenarios.

Scenario 1 – Employment Contract

A startup founder enters details about a new hire and selects an employment contract. The system generates a structured agreement containing sections for roles, responsibilities, compensation and confidentiality. The user can customize the wording and download a branded PDF.

Scenario 2 – Non-Disclosure Agreement

A freelancer can provide the parties, confidentiality scope and effective date for an NDA. LegalEase uses those inputs to create a structured draft that can be reviewed and edited before export.

Scenario 3 – Residential Lease Agreement

A landlord can provide property, tenant and lease information. The resulting document can be formatted and exported as a DOCX file for further editing.

Common User Flow

- Select or enter the type of legal document.
- Enter the parties involved.
- Enter terms and conditions.
- Provide the effective date when required.
- Generate the document using the AI backend.
- Review the generated content.
- Edit the content if necessary.
- Download the final draft as TXT, DOCX or PDF.

3. Pre-requisites

The source documentation identifies the following knowledge and setup requirements.

Requirement	Purpose
FastAPI knowledge	Build and understand the backend API.
Google Gemini API familiarity	Connect the AI generation layer.
Streamlit knowledge	Build the user interface.
Python proficiency	Develop and maintain the application.
Environment setup	Install and manage required Python packages.

Core dependencies used by the implemented project include:

- fastapi and uvicorn[standard] for the backend service.
- streamlit for the frontend.
- google-genai for Gemini integration.
- python-docx for DOCX generation.
- fpdf2 for PDF generation.
- Pillow for image/logo handling.
- python-dotenv for environment variables.
- requests and httpx for HTTP operations and testing.
- pydantic for request validation.
- pytest for automated testing.

Environment Variables

The Gemini API key is stored through environment configuration rather than being placed directly in source code. The local .env file is excluded from Git through .gitignore.

4. Project Workflow

The project follows the milestone-oriented workflow of the source documentation.

Activity	Main Work
Activity 1	Model selection and system architecture
Activity 2	Core document generation, editing and export
Activity 3	FastAPI main.py and API logic
Activity 4	Streamlit frontend and dynamic preview
Activity 5	Deployment and environment configuration
Activity 6	Testing, verification and conclusion

End-to-End Flow

User Input → Streamlit → FastAPI → Pydantic Validation → Gemini AI → Generated Draft → Editable Preview → Formatter → TXT/DOCX/PDF Download

The frontend does not need to implement the AI logic itself. It sends document details to the backend, while the backend coordinates validation and generation. Formatting utilities then convert the generated text into downloadable document formats.

Key Design Principle

The architecture separates user interface, API logic, AI generation and document formatting. This makes each major responsibility easier to maintain and test independently.

5. Milestone 1 – Model Selection and System Architecture

The source documentation originally describes research around Gemini 1.5 Pro. During actual implementation, the available Gemini configuration was updated to **gemini-3.5-flash-lite**. This report records the model used by the implemented application rather than the historical model name in the source draft.

Activity 1.1 – AI Model Requirements

- Generate structured documents from user inputs.
- Handle document type, parties, terms and dates.
- Produce coherent sections and clauses.
- Return text that can be edited and formatted by the application.

Activity 1.2 – Architecture Components

- Frontend: Streamlit interface for input, preview, editing and downloads.
- Backend: FastAPI service exposing document generation routes.
- AI Core: Gemini integration for document generation.
- Formatting: DOCX, PDF and TXT output utilities.
- Configuration: environment variables for API credentials and deployment settings.

Architecture Diagram

Layer	Responsibility
Streamlit UI	Collect input, display output, edit and download
FastAPI	Validate requests and coordinate generation
Pydantic	Validate document request fields
Gemini Generator	Create the legal document draft
Formatting Utilities	Produce TXT, DOCX and PDF
Render	Host the deployed frontend and backend

6. Development Environment and Project Structure

The application was developed in Python using an isolated virtual environment. The project contains separate folders for AI generation, backend routes/schemas, document utilities, frontend code, assets and tests.

Project Structure

```
LegalEase/ ■■■ ai_core/ ■ ■■■ __init__.py ■ ■■■ gemini_generator.py ■■■ assets/ ■ ■■■
.gitkeep ■ ■■■ logo.png ■■■ backend/ ■ ■■■ __init__.py ■ ■■■ main.py ■ ■■■ routes.py ■
■■■ schemas.py ■■■ document_utils/ ■ ■■■ __init__.py ■ ■■■ exporters.py ■ ■■■
formatting.py ■■■ frontend/ ■ ■■■ app.py ■■■ tests/ ■ ■■■ __init__.py ■ ■■■ test_api.py
■ ■■■ test_exporters.py ■■■ .env ■■■ .env.example ■■■ .gitignore ■■■ Dockerfile ■■■
docker-compose.yml ■■■ Makefile ■■■ Procfile ■■■ render.yaml ■■■ requirements.txt ■■■
README.md
```

Important Modules

Module	Purpose
ai_core/gemini_generator.py	Gemini model configuration and generation.
backend/main.py	FastAPI application initialization.
backend/routes.py	Generation and export API routes.
backend/schemas.py	Pydantic request validation.
frontend/app.py	Streamlit user interface.
document_utils/formatting.py	Text/preview formatting helpers.
document_utils/exporters.py	TXT/DOCX/PDF document output.
tests/	Automated API and exporter tests.

7. Milestone 2 – Core Functionalities

Activity 2.1 – Legal Document Generation

The application accepts a document type, parties, terms and optional date information. These values are passed through the backend to the Gemini generator, which returns a structured draft for display in the frontend.

Core Features

- AI-assisted legal document draft generation.
- Editable generated text.
- Professional headings and sections.
- Terms represented clearly in the output.
- TXT, DOCX and PDF export.
- Logo and branding support for document outputs.
- Placeholder handling when required information is missing.

Activity 2.2 – FastAPI Backend

FastAPI acts as the bridge between the Streamlit interface and the Gemini generation layer. The backend receives validated JSON input, invokes the generator and returns the generated content.

Example request fields

Field	Description
document_type	Type of document to generate.
parties	People or organizations involved.
terms	Terms and conditions supplied by the user.
dates	Effective date or related date information.

8. Document Formatting and Export

The original documentation emphasizes converting raw AI-generated text into structured files with legal formatting, logos, tables and footers. The implemented application provides the same core output workflow.

Formatting Utilities

Function	Purpose
<code>sanitize_text(text)</code>	Cleans generated text for consistent formatting.
<code>format_docx(text, doc_type)</code>	Creates a formatted Word document.
<code>format_pdf(text, doc_type)</code>	Creates a branded PDF.
<code>format_html_preview(text)</code>	Creates styled preview content for Streamlit.

DOCX Output

- Uses python-docx.
- Supports title and paragraph formatting.
- Can embed the LegalEase logo.
- Can represent terms in a table.
- Provides a structured Word document for further editing.

PDF Output

- Uses fpdf2.
- Supports branded output with the LegalEase logo.
- Uses formatted headings and terms.
- Supports footer information across generated pages.

TXT Output

TXT provides the simplest export option and preserves the generated document as plain text.

9. Milestone 3 – API Logic Integration

The API layer provides a modular bridge between the frontend and AI core. The source documentation specifies a FastAPI application, a Pydantic request model and a POST /generate endpoint.

Activity 3.1 – main.py

- Initializes the FastAPI application.
- Provides the root service route.
- Provides the health-check route.
- Includes the modular router defined in routes.py.
- Configures CORS for frontend/backend communication.

Activity 3.2 – routes.py

- Receives document generation requests.
- Uses validated request data.
- Calls the Gemini document generator.
- Returns generated text to the frontend.
- Supports export-related processing.

API Endpoints

Method	Endpoint	Purpose
GET	/	Basic service/root response.
GET	/health	Health and configuration status.
POST	/generate	Generate a legal document draft.
POST	/generate/export	Generate and prepare an export.

Validation

Pydantic schemas are used to validate incoming document information before the generation workflow proceeds. This reduces malformed requests and keeps the API contract explicit.

10. Backend Components

FastAPI Application

The backend is organized into a main application module, routes and schemas. This structure keeps application startup, request handling and validation separate.

Gemini Generator

The AI core uses the Google GenAI SDK. The model name is read from configuration and the generation request uses a controlled prompt and generation settings suitable for structured document drafting.

Health Check

The /health endpoint reports the service status, whether Gemini configuration is present, and the configured model name. This was used during local and cloud deployment verification.

Backend Runtime

The deployed backend runs as a web service using Uvicorn/FastAPI. The production deployment uses Render and exposes the backend through its public service URL.

Backend Responsibility Summary

Responsibility	Implementation
HTTP server	FastAPI + Uvicorn
Request validation	Pydantic
AI generation	Google GenAI / Gemini
Export coordination	FastAPI export route
Health monitoring	GET /health
CORS	FastAPI middleware configuration

11. Milestone 4 – Frontend Development

The Streamlit frontend provides the user-facing workflow. The source documentation specifies a responsive interface with a logo, input controls, generation, preview, editing and downloads.

User Interface Elements

- Legal document type input.
- Parties involved input.
- Terms and conditions input.
- Effective date/jurisdiction-related information as supported by the application.
- Generate Document action.
- Generated document preview.
- Edit Document workflow.
- Download buttons for TXT, DOCX and PDF.
- LegalEase logo and branding.

Frontend-to-Backend Communication

When the user selects Generate Document, the Streamlit application sends a POST request to the FastAPI backend. The backend generates the content through Gemini and returns the result.

Preview

Generated text is sanitized and rendered in a styled preview area. The interface also allows the user to switch to an editable representation before exporting.

Branding

The LegalEase logo is displayed in the application and can also be embedded in generated PDF and DOCX outputs.

12. Dynamic Preview and Editing

Dynamic Generation

The preview is updated after a successful backend response. This allows the user to inspect the generated legal draft without immediately downloading a file.

Editing Workflow

- Generate the initial draft.
- Review the generated text.
- Select the edit option.
- Modify the generated wording in the editable area.
- Use the updated content for download.

Why Editing is Included

The source documentation specifically includes editable document preview as a core feature. The editing step allows users to correct, customize or refine the generated draft before saving it.

Output Consistency

The same edited content is used as the basis for the available download formats so that the export reflects the version reviewed by the user.

Example Workflow

Step	Action
1	Enter Employment Contract details.
2	Click Generate Document.
3	Review generated clauses.
4	Edit wording if necessary.
5	Download TXT, DOCX or PDF.

13. AI Integration and Prompting

The AI core is responsible for transforming structured user information into a legal document draft. The application uses Google Gemini through the Google GenAI SDK.

Generation Configuration

Setting	Implemented Configuration
SDK	google-genai
Model	gemini-3.5-flash-lite
Temperature	0.2
Maximum output tokens	8000
Input	Document type, parties, terms and related details
Output	Structured legal document draft

Prompting Principles

- Produce professional legal document drafts.
- Use the user-provided information as the source of document facts.
- Avoid inventing missing factual information.
- Use placeholders where information has not been supplied.
- Organize content with clear headings and sections.
- Include signature blocks where appropriate.
- Do not present the generated content as a substitute for professional legal advice.

Model Update Note

The original source PDF refers to Gemini 1.5 Pro. The implemented deployed project uses gemini-3.5-flash-lite. This is documented here so the report reflects the actual application.

14. Milestone 5 – Deployment

The source documentation describes preparing the application for deployment by installing production dependencies, configuring environment variables and testing the complete workflow. The implemented project additionally uses Docker/Render deployment configuration.

Deployment Preparation

- Create and activate a Python virtual environment.
- Install dependencies from requirements.txt.
- Configure the Gemini API key through environment variables.
- Prepare FastAPI and Streamlit services for deployment.
- Include deployment configuration such as Dockerfile, Procfile and render.yaml.

Cloud Deployment

Service	Purpose	Current URL
LegalEase Frontend	Streamlit application	https://legalease-frontend-w4gl.onrender.com
LegalEase Backend	FastAPI API service	https://legalease-backend-v2.onrender.com

Deployment Verification

- Backend health endpoint was verified.
- Frontend successfully connected to the deployed backend.
- A sample legal document was generated.
- TXT, DOCX and PDF downloads were tested.
- Logo rendering in the generated PDF was verified.

15. Testing and Verification

Testing was performed at both automated and end-to-end levels. The project contains tests for API behavior and document exporters.

Automated Tests

The project test suite previously completed successfully with **7 passed**. The warnings reported during that run were Python 3.14 deprecation warnings originating from a dependency and were not test failures.

Functional Verification

- Backend health response verified.
- POST /generate behavior verified.
- Frontend generation flow verified.
- Generated document preview verified.
- TXT download verified.
- DOCX download verified.
- PDF download verified.
- PDF logo rendering verified.

Sample Test Data

Field	Sample
Document type	Employment Contract
Parties	Jane Doe (Service Provider), TechNova Inc. (Client)
Terms	Payment within 30 days; delivery by agreed deadline; confidentiality; 15-day termination notice
Jurisdiction	India

Result

The deployed application generated the sample document successfully and produced downloadable TXT, DOCX and PDF files. The branded PDF was also visually checked for logo placement and formatting.

16. Exploring LegalEase – User Inputs

1. Document Type

The document type determines the general structure of the generated draft. Examples from the source documentation include Agreement, Contract, NDA, Lease Agreement and Employment Offer Letter.

2. Parties Involved

The parties field identifies the people or organizations involved in the document. The source examples include a freelancer and client or a tenant and landlord.

3. Terms and Conditions

Terms describe the specific clauses or rules requested by the user. The source workflow uses semicolon-separated terms so that they can be represented as distinct items or table rows.

Example Terms

- Payment to be made within 30 days of invoice.
- The provider agrees to deliver work by the agreed deadline.
- Confidentiality must be maintained.
- Either party may terminate with 15 days notice.

4. Effective Date

The effective date indicates when the agreement is intended to take effect. Example formats in the source include April 10, 2025 and 10/04/2025.

Practical Input Flow

The user provides the relevant fields, submits the generation request and then reviews the result before deciding whether to edit or download it.

17. Generated Document and Downloads

Document Generated

After the AI generation request completes, the generated document is shown in the application preview. The user can inspect the structure and content before exporting.

Editing the Document

The source documentation describes an Edit Document action that opens an editable area. The implemented Streamlit application provides an editable text workflow so the generated content can be changed before export.

Download Formats

Format	Use
TXT	Simple plain-text copy of the document.
DOCX	Editable Word document with formatting and branding.
PDF	Formatted and branded document suitable for presentation/printing.

PDF Branding

The generated PDF can include the LegalEase logo and footer formatting. The deployed PDF output was tested with the project logo and verified visually.

DOCX Branding

The DOCX formatter supports logo embedding, headings, paragraph formatting and terms-table generation based on the document content.

18. Security and Environment Configuration

The application uses an environment variable approach for the Gemini API credential. The credential is not intended to be stored in the public source repository.

Configuration Files

File	Purpose
.env	Local secret/configuration values; excluded from Git.
.env.example	Example environment configuration without the secret.
.gitignore	Prevents secrets, virtual environments and caches from being committed.
render.yaml	Render deployment configuration.
Dockerfile	Container build configuration.
requirements.txt	Python dependency specification.

Repository Safety

The project repository excludes the local .env file. The application configuration therefore keeps the actual Gemini credential outside the committed source files.

Legal/Privacy Consideration

Because legal documents may contain sensitive information, users should avoid entering data that they are not authorized to process and should review generated content before use.

19. Deployment Architecture

Production Architecture

Browser → Render Streamlit Frontend → Render FastAPI Backend → Google Gemini API

Component Responsibilities

Component	Role
User Browser	Provides the interactive web experience.
Render Frontend	Hosts Streamlit UI and sends API requests.
Render Backend	Hosts FastAPI generation/export routes.
Gemini API	Generates the document draft from structured input.
Document Utilities	Create TXT, DOCX and PDF output.

Local Development Architecture

During local development, the frontend and backend can run as separate processes. The frontend communicates with the backend through the configured backend URL.

Deployment Files

- Dockerfile for containerized deployment.
- docker-compose.yml for local multi-service configuration.
- Procfile for process definition.
- render.yaml for Render configuration.
- requirements.txt for dependency installation.

20. Team Contribution

LegalEase was developed as a five-member NAAN MUDHALVAN group project. Guna K served as Team Leader and coordinated the development and integration work.

No.	Team Member	Role
1	Guna K	Team Leader
2	Vinith Kumar S	Team Member
3	Lokesh S	Team Member
4	Bala S	Team Member
5	Sanjay K	Team Member

Team Leader Contribution

- Project coordination and task organization.
- Application architecture and module integration.
- Gemini AI integration and backend/frontend connection.
- Testing of document generation and exports.
- Deployment and production verification.
- Project documentation and repository maintenance.

Project Management

The project was developed incrementally: architecture first, followed by core functionality, API integration, frontend development, testing and deployment. This follows the milestone organization of the source documentation.

21. Limitations and Future Enhancements

Current Scope

- The system focuses on AI-assisted document drafting rather than providing legal representation.
- The generated content should be reviewed by the user before practical use.
- The current application does not include a full user account/database history system.
- The current template set can be expanded for additional document categories.

Future Enhancements

- User authentication and role-based access.
- Document history and saved drafts.
- Database-backed document storage.
- More legal document templates.
- Multilingual document generation.
- Advanced document comparison and clause analysis.
- Integration with approved legal reference databases.
- More comprehensive automated and end-to-end testing.
- Expanded document management and collaboration features.

Scope for Scalability

The separation of frontend, backend, AI generation and formatting modules provides a foundation for extending the application without replacing the complete architecture.

22. Conclusion

LegalEase demonstrates an end-to-end AI application workflow for legal document drafting. The project combines a Streamlit interface, FastAPI backend, Gemini AI generation and document export utilities into one integrated application.

The project follows the milestone structure of model selection, architecture, core functionality, API integration, frontend development, deployment and testing. Users can provide document details, generate a draft, review and edit it, and download the result in TXT, DOCX or PDF format.

The deployed implementation was tested through an actual generation workflow, including the download of all supported formats and verification of PDF branding. This confirms the practical end-to-end operation of the implemented system.

Final Project Summary

Area	Implemented Result
AI Generation	Gemini-powered legal document drafting
Frontend	Streamlit interface with preview and editing
Backend	FastAPI API with validation
Exports	TXT, DOCX and PDF
Branding	LegalEase logo and document formatting
Testing	Automated tests plus end-to-end verification
Deployment	Frontend and backend deployed on Render

23. Disclaimer

LegalEase is an AI-assisted document drafting application. The generated content is provided for drafting and informational purposes and should be reviewed for accuracy and suitability before use. LegalEase does not replace advice, review or representation by a qualified legal professional.

Project Details

Project	LegalEase: AI-Powered Legal Document Generator
Programme	NAAN MUDHALVAN
Team Leader	Guna K
Team Size	5
Technology	Python, Streamlit, FastAPI, Google Gemini
Deployment	Render

End of Project Documentation